Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 8

Disciplina: Tehnici de programare

Tema: “Github, pas cu pas”

A efectuat : st. gr. SI–262
Inculeț Andrian

A verificat : asistent universitar
Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

A1. Contul GitHub (10 min)

Dacă ai deja cont GitHub, autentifică-te și sari la A2.

1. Deschide în browser github.com și apasă Sign up.
2. Scrie adresa de e-mail pe care o citești zilnic (poate fi cea de student).
3. Alege o parolă și un nume de utilizator (username). Numele apare în toate linkurile tale: alege ceva profesional, de exemplu ion-popescu sau ipopescu, nu xX_hacker_Xx.
4. Rezolvă verificarea și confirmă codul primit pe e-mail.
5. Notează-ți numele de utilizator: îl folosești imediat.

De observat. GitHub îți poate cere mai târziu autentificarea în doi pași (2FA). Activeaz-o când ți-o cere, cu o aplicație de autentificare pe telefon: e același principiu pe care îl recomanzi oricui ca specialist în securitate.

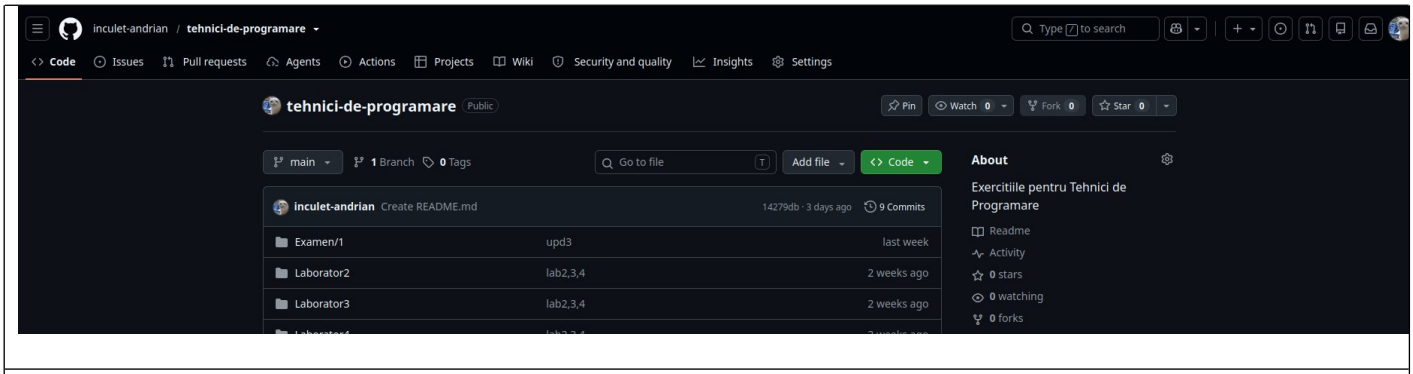


Fig 1.1 – Profilul Github deja existent + un screenshot al repository pentru TP

A2. Git pe calculator: verifici și te prezinți (10 min)

Deschide terminalul (PowerShell pe Windows sau terminalul pe Linux) și verifică dacă git e instalat.

```
git --version
```

Pe ecran trebuie să vezi:

```
git version 2.53.0.windows.1      (numărul poate fi altul, nu contează)
```

Dacă vezi „git is not recognized" sau „command not found", git nu e instalat: pe Windows rulează winget install Git.Git, închide și redeschide terminalul; pe Linux, sudo apt install git.

Acum spui o singură dată cine ești. Pune numele tău real și EXACT adresa de e-mail din contul GitHub: așa GitHub leagă commit-urile de contul tău.

```
git config --global user.name "Ion Popescu"
git config --global user.email "adresa-ta@exemplu.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

Verifici ce ai setat:

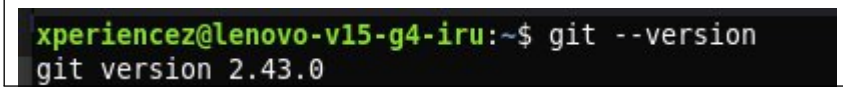
```
git config --global --list
```

Pe ecran trebuie să vezi:

```
user.name=Ion Popescu
```

```
user.email=adresa-ta@exemplu.com
init.defaultbranch=main
pull.rebase=false
```

Ce fac ultimele două linii. init.defaultBranch main numește ramura principală „main", ca pe GitHub. pull.rebase false îi spune lui git cum să unească schimbările la pull, ca să nu te întrebe mai târziu.

Comanda:
`git --version`
Output:
`git version 2.43.0`   

Comanda:

`git config --global user.name "Inculeț Andrian"`
`git config --global user.email "andrian.inculet@stud.fcim.utm.md"`
`git config --global init.defaultBranch main`
`git config --global pull.rebase false`
 	
`git config --global --list`

Output:

`filter.lfs.smudge=git-lfs smudge -- %f`
`filter.lfs.process=git-lfs filter-process`
`filter.lfs.required=true`
`filter.lfs.clean=git-lfs clean -- %f`
`user.name=Inculeț Andrian`
`user.email=andrian.inculet@stud.fcim.utm.md`
`init.defaultbranch=main`
`pull.rebase=false`

```
xperiencez@lenovo-v15-g4-iru:~$ git config --global --list
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
filter.lfs.clean=git-lfs clean -- %f
user.name=Inculet Andrian
user.email=andrian.inculet@stud.fcim.utm.md
xperiencez@lenovo-v15-g4-iru:~$
xperiencez@lenovo-v15-g4-iru:~$ git config --global user.name "Inculeț Andrian"
xperiencez@lenovo-v15-g4-iru:~$ git config --global user.email "andrian.inculet@
stud.fcim.utm.md"
xperiencez@lenovo-v15-g4-iru:~$ git config --global init.defaultBranch main
xperiencez@lenovo-v15-g4-iru:~$ git config --global pull.rebase false
xperiencez@lenovo-v15-g4-iru:~$ git config --global --list
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
filter.lfs.clean=git-lfs clean -- %f
user.name=Inculeț Andrian
user.email=andrian.inculet@stud.fcim.utm.md
init.defaultbranch=main
pull.rebase=false
xperiencez@lenovo-v15-g4-iru:~$ ^C
xperiencez@lenovo-v15-g4-iru:~$
```

A3. Primul depozit, creat pe GitHub (8 min)

În browser, pe GitHub:

1. Sus în dreapta apasă semnul + și alege New repository.
2. La Repository name scrie: exercitiu-git
3. La Description (opțional): Exercitii de git pentru Tehnici de programare
4. Alege Public.
5. Bifează Add a README file.
6. Apasă Create repository.

Ești acum pe pagina depozitului, la adresa https://github.com/<numele-tău>/exercitiu-git, și vezi fișierul README.md.

Nota: deja consideram exercitiu-git ca si repozitorul tehnici-de-programare

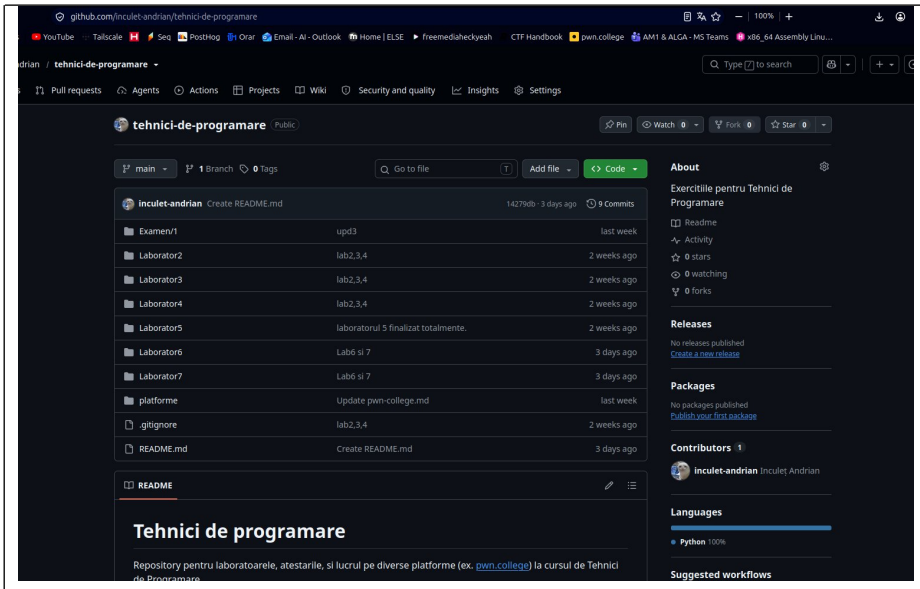


Figura 1.2 – Repozitoriul TP

A4. Îl aduci pe calculator: git clone (8 min)

Pe pagina depozitului apasă butonul verde Code, rămâi pe fila HTTPS și copiază adresa (se termină în .git). Apoi, în terminal, mergi într-un folder de lucru și clonează:

```
cd ~
git clone https://github.com/<numele-tau>/exercitiu-git.git
cd exercitiu-git
dir       # Linux: ls
```

Pe ecran trebuie să vezi:

```
Cloning into 'exercitiu-git'...
...
README.md
```

De observat. De acum lucrezi ÎN folderul exercitiu-git. Toate comenzile git de mai jos se dau de aici; dacă ieși din folder, git nu mai știe despre ce depozit vorbești.

Comanda:

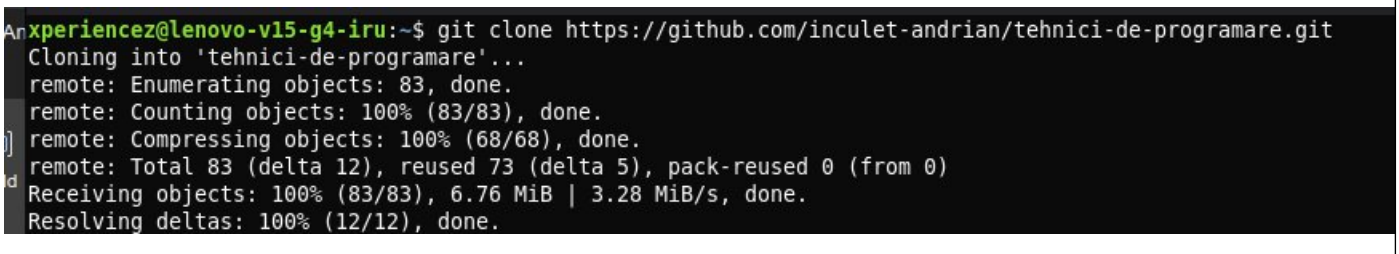
```
cd ~

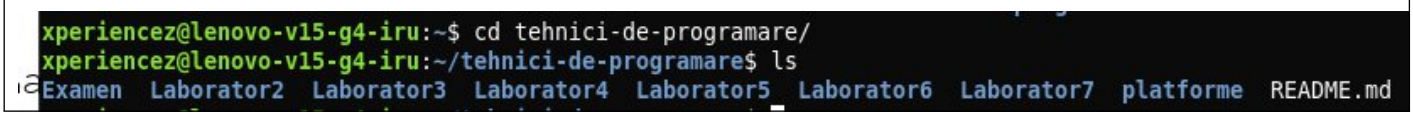

git clone https://github.com/inculet-andrian/tehnici-de-programare.git
cd tehnici-de-programare/
```

Output:

```
Cloning into 'tehnici-de-programare'...
remote: Enumerating objects: 83, done.
remote: Counting objects: 100% (83/83), done.
remote: Compressing objects: 100% (68/68), done.
remote: Total 83 (delta 12), reused 73 (delta 5), pack-reused 0 (from
0)
Receiving objects: 100% (83/83), 6.76 MiB | 3.28 MiB/s, done.
Resolving deltas: 100% (12/12), done.


Examen  Laborator2  Laborator3  Laborator4  Laborator5  Laborator6
Laborator7  platforme  README.md
```





A5. Prima ta fotografie: status, add, commit (12 min)

Creează un fișier nou, salut.py, din editor (nu cu echo din PowerShell, care scrie fișierul într-o codare pe care Python nu o citește). Scrie în el un singur rând, print("Salut, GitHub!"), salvează cu Ctrl+S, apoi rulează-l:

```
code salut.py      # sau deschizi folderul in editorul tau
python salut.py
```

Pe ecran trebuie să vezi:

```
Salut, GitHub!
```

Întreabă git ce vede:

```
git status
```

Pe ecran trebuie să vezi:

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        salut.py
```

„Untracked" înseamnă: git vede fișierul, dar încă nu îl urmărește. Îl pui în fotografie cu add, apoi întrebi din nou:

```
git add salut.py
git status
```

Pe ecran trebuie să vezi:

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   salut.py
```

Fișierul e pregătit. Acum faci fotografia, cu un mesaj care spune ce ai schimbat:

```
git commit -m "Adaug salut.py"
```

Pe ecran trebuie să vezi:

```
[main aaf9ebc] Adaug salut.py
 1 file changed, 1 insertion(+)
 create mode 100644 salut.py
```

De observat. aaf9ebc e codul fotografiei; la tine e altul. Un avertisment de tipul „LF will be replaced by CRLF" e normal pe Windows și nu e o eroare.

Comanda:
`python3 salut.py`
Output:
`Salut, GitHub!` 

Comanda:

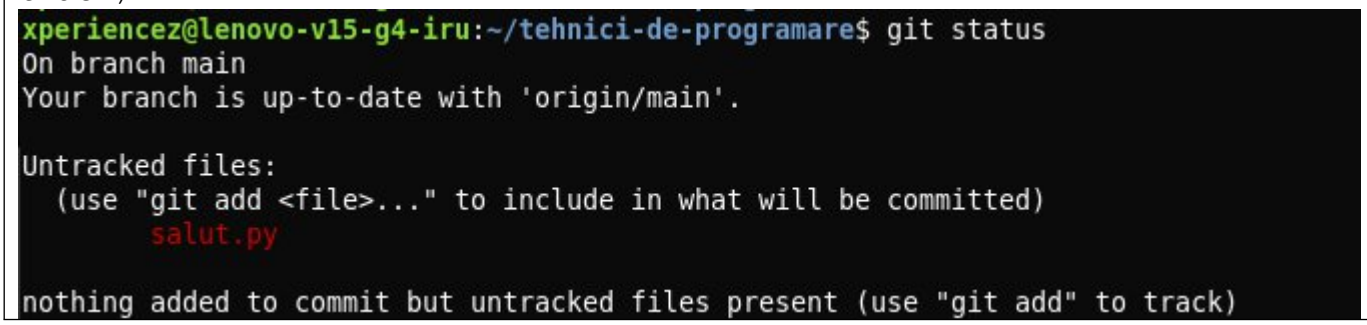
```
git status
```

Output:

```
On branch main
Your branch is up-to-date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        salut.py

nothing added to commit but untracked files present (use "git add" to
track)
```



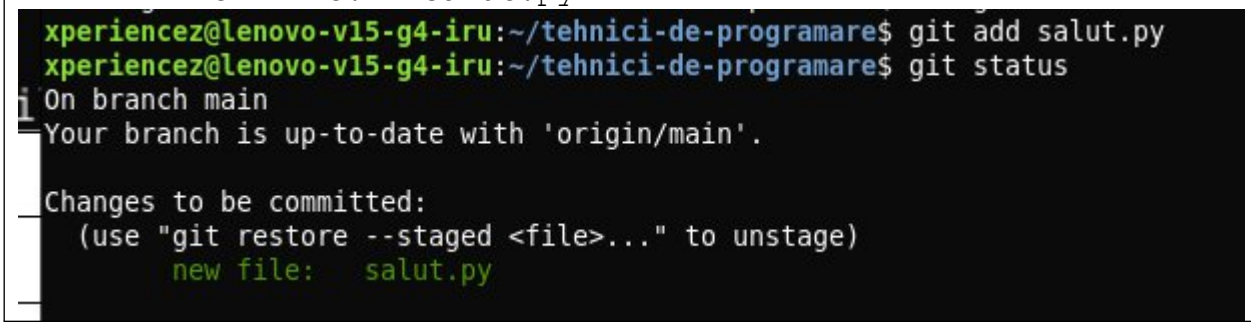
Comanda:

```
git add salut.py
git status
```

Output:

```
On branch main
Your branch is up-to-date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   salut.py
```



Comanda:

```
git commit -m "Adaug salut.py"
```

Output:

```
[main 394f57e] Adaug salut.py
```

```
 1 file changed, 1 insertion(+)
 create mode 100644 salut.py
xperiencez@lenovo-v15-g4-iru:~/tehnici-de-programare$ git commit -m "Adaug salut.py"
[main 394f57e] Adaug salut.py
 1 file changed, 1 insertion(+)
 create mode 100644 salut.py
```

A6. O trimiți pe GitHub: git push (12 min)

```
git push
```

Prima dată, git îți cere să te autentifici la GitHub:

Pe Windows: se deschide o fereastră „Connect to GitHub". Alege Sign in with your browser, autentifică-te în browser și apasă Authorize. Fereastra se închide singură, iar push-ul continuă. Data viitoare nu te mai întreabă.

Pe Linux: când îți cere Username, scrii numele de utilizator GitHub. Când îți cere Password, NU merge parola contului: îți trebuie un token. Pe GitHub: poza ta (dreapta sus) → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic), bifezi repo, Generate, copiezi tokenul și îl lipești la Password. Păstrează-l ca pe o parolă.

Pe ecran trebuie să vezi:

```
To https://github.com/<numele-tau>/exercitiu-git.git
  e75c0d3..aaf9ebc  main -> main
```

Reîncarcă pagina depozitului în browser: apare salut.py, iar lângă el mesajul „Adaug salut.py".

Comanda:

```
git push
```

Output:

```
Username for 'https://github.com': inculet-andrian
Password for 'https://inculet-andrian@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 314 bytes | 314.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/inculet-andrian/tehnici-de-programare.git
   14279db..394f57e  main -> main
```

```
xperiencez@lenovo-v15-g4-iru:~/tehnici-de-programare$ git push
Username for 'https://github.com': inculet-andrian
Password for 'https://inculet-andrian@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 314 bytes | 314.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/inculet-andrian/tehnici-de-programare.git
   14279db..394f57e  main -> main
```

A7. O schimbare făcută pe GitHub, adusă înapoi: git pull (8 min)

În browser, deschide README.md, apasă creionul (Edit this file) și adaugă un rând, de exemplu: Aici exersez git. Jos apasă Commit changes, apoi din nou Commit changes. Acum GitHub are o fotografie pe care calculatorul tău nu o are. O aduci:

```
git pull
type README.md     # Linux: cat README.md
```

Pe ecran trebuie să vezi:

```
Updating aaf9ebc..1c2d3e4
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)

# exercitiu-git
Aici exersez git.
```

Regula de aur. Înainte să începi lucrul, git pull. După ce ai terminat, git push. Așa calculatorul și GitHub rămân la fel.

Comanda:
`git pull` `cat README.md`
Output: (deja aveam un README.md creat)

`# Tehnici de programare`
 	
`Repository pentru laboratoarele, atestarile, si lucrul pe diverse platforme (ex. [pwn.college](https://pwn.college/hacker/211629)) la cursul de Tehnici de Programar`
`...`



A8. Ce NU urcă pe GitHub: .gitignore (8 min)

Mediul virtual (.venv) are mii de fișiere care nu au ce căuta în depozit. Creează-l și vezi ce crede git:

```
python -m venv .venv
git status --short
```

Pe ecran trebuie să vezi:

```
?? .venv/
```

Dacă nu apare nimic, Python-ul tău (3.13 sau mai nou) a pus singur o regulă în .venv; faci oricum pasul, fiindcă .gitignore protejează și alte fișiere. Creează din editor un fișier numit exact .gitignore, cu două rânduri, și întreabă din nou:

```
# continutul fisierului .gitignore:
.venv/
__pycache__/

git status --short
```

Pe ecran trebuie să vezi:

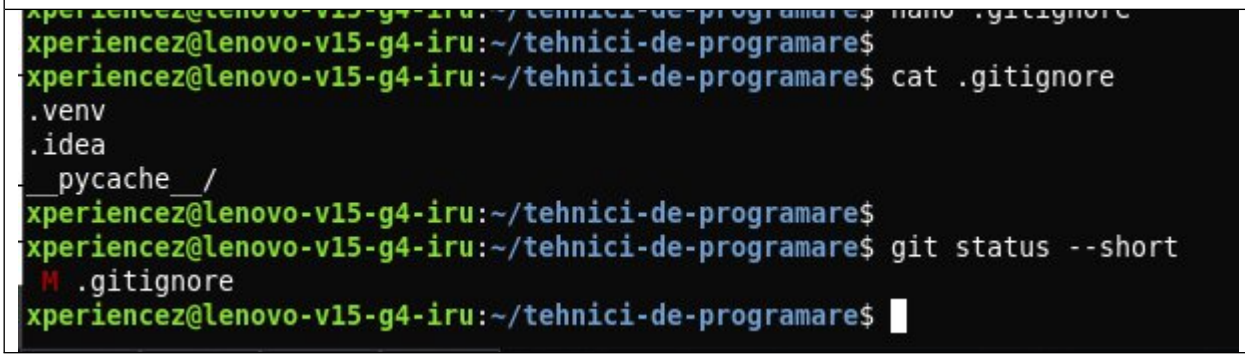
```
?? .gitignore
```

.venv a dispărut din listă: git îl ignoră. Salvezi și trimiți regula:

```
git add .gitignore
git commit -m "Ignor mediul virtual"
git push
```

Comanda:
`python -m venv .venv` `git status --short`
Output:
(nimic)

Comanda:
`git status --short`
Output:
M .gitignore

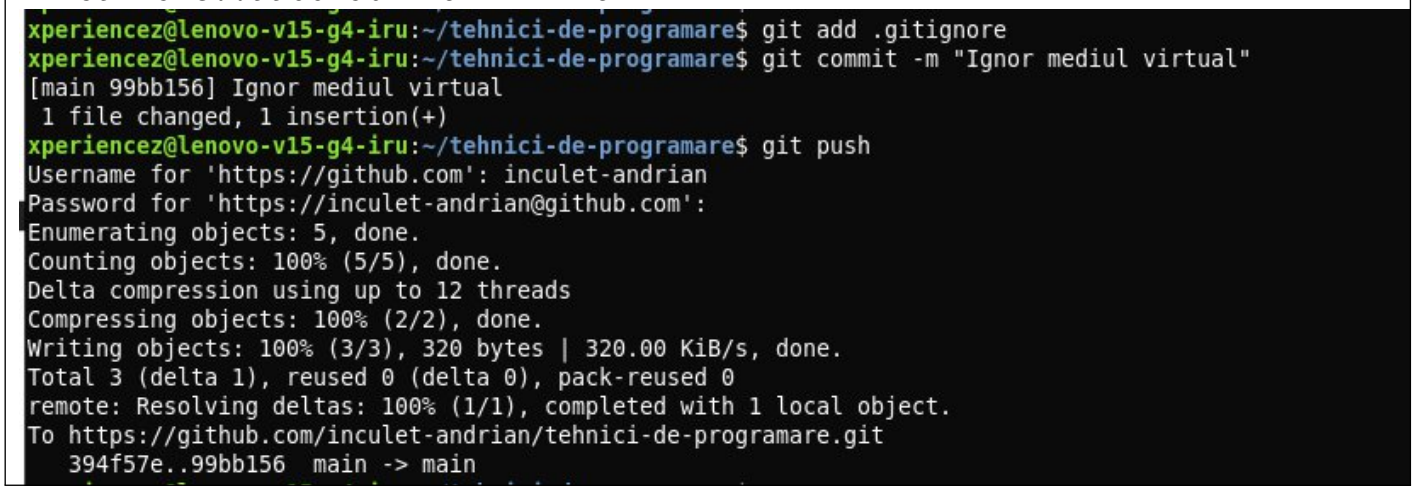


Comanda:

```
git add .gitignore
git commit -m "Ignor mediul virtual"
git push
```

Output:

```
Username for 'https://github.com': inculet-andrian
Password for 'https://inculet-andrian@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 320 bytes | 320.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/inculet-andrian/tehnici-de-programare.git
   394f57e..99bb156  main -> main
```



A9. Citești istoria și schimbările: log și diff (8 min)

```
git log --oneline
```

Pe ecran trebuie să vezi:

```
4f5a6b7 Ignor mediul virtual
1c2d3e4 Update README.md
aaf9ebc Adaug salut.py
e75c0d3 Initial commit
```

Adaugă un al doilea rând în salut.py, de exemplu print("al doilea"), și întreabă ce s-a schimbat, ÎNAINTE de commit:

```
git diff
```

Pe ecran trebuie să vezi:

```
--- a/salut.py
+++ b/salut.py
```

```
@@ -1 +1,2 @@
 print("Salut, GitHub!")
+print("al doilea")
```

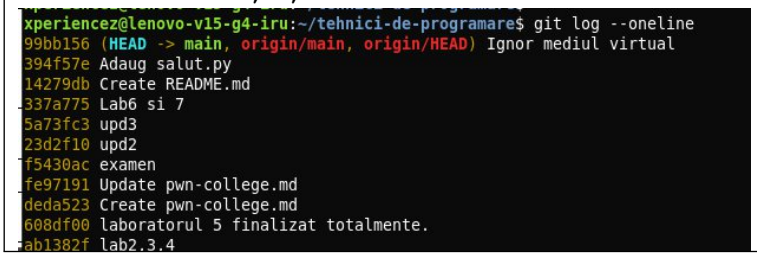
Rândul cu + e adăugat, un rând cu - ar fi fost șters. Te-ai răzgândit? Întorci fișierul la ultima fotografie:

```
git restore salut.py
git status
```

Pe ecran trebuie să vezi:

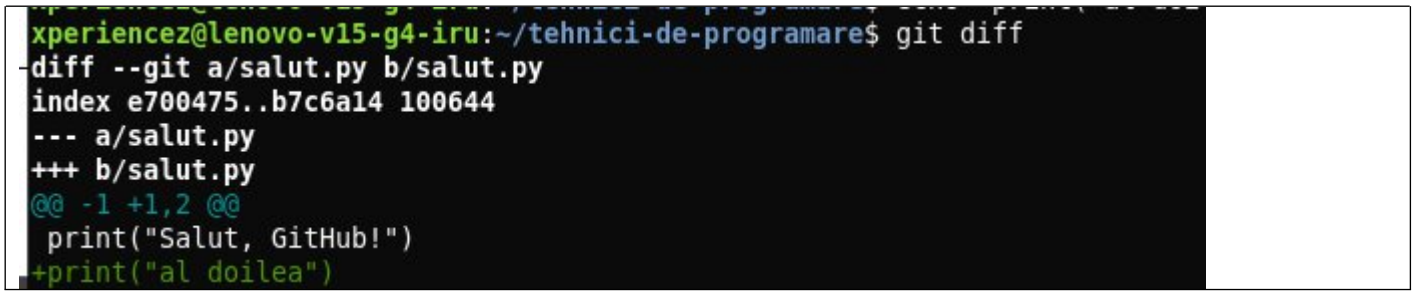
```
nothing to commit, working tree clean
```

Comanda:
`git log --oneline`
Output:

`99bb156 (HEAD -> main, origin/main, origin/HEAD) Ignor mediul virtual`
`394f57e Adaug salut.py`
`14279db Create README.md`
`337a775 Lab6 si 7`
`5a73fc3 upd3`
`23d2f10 upd2`
`f5430ac examen`
`fe97191 Update pwn-college.md`
`deda523 Create pwn-college.md`
`608df00 laboratorul 5 finalizat totalmente.`
`ab1382f lab2,3,4`
 

Comanda:
`git diff`
Output:

`diff --git a/salut.py b/salut.py`
`index e700475..b7c6a14 100644`
`--- a/salut.py`
`+++ b/salut.py`
`@@ -1 +1,2 @@`
` print("Salut, GitHub!")`
`+print("al doilea")`



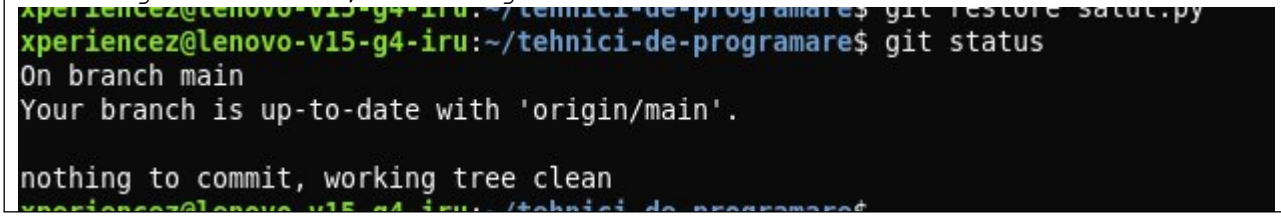
Comanda:

```
git restore salut.py
git status
```

Output:

```
On branch main
Your branch is up-to-date with 'origin/main'.

nothing to commit, working tree clean
```



A10. Al doilea depozit, pornit de pe calculator (10 min)

De data asta pornești invers: întâi folderul local, apoi depozitul de pe GitHub. Ieși din exercitiu-git și creează un folder nou:

```
cd ~
mkdir exercitiu-2
cd exercitiu-2
git init
code README.md      # scrii un rand: # Al doilea depozit, si salvezi
git add README.md
git commit -m "Primul commit"
```

Pe GitHub creează un depozit nou exercitiu-2, Public, dar de data asta NU bifa Add a README file. GitHub îți arată o pagină cu instrucțiuni; folosești secțiunea „push an existing repository":

```
git remote add origin https://github.com/<numele-tau>/exercitiu-2.git
git branch -M main
git push -u origin main
```

Pe ecran trebuie să vezi:

```
branch 'main' set up to track 'origin/main'.
```

De observat. -u leagă ramura ta de cea de pe GitHub o singură dată; de acum încolo îți ajunge git push simplu.

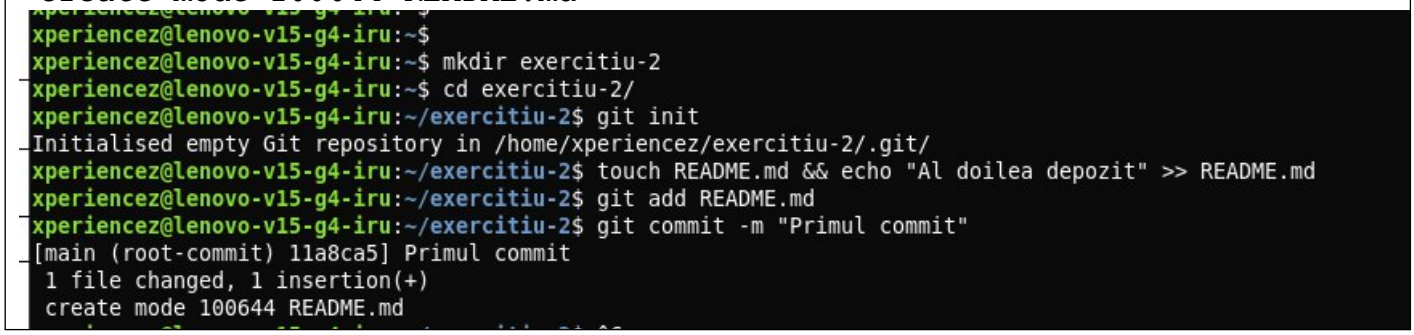
Comanda:

```
cd ~
mkdir exercitiu-2
cd exercitiu-2
git init
touch README.md && echo "Al doilea depozit" >> README.md
git add README.md
git commit -m "Primul commit"
```

Output:

```
Initialised empty Git repository in /home/xperiencez/exercitiu-2/.git/

[main (root-commit) 11a8ca5] Primul commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

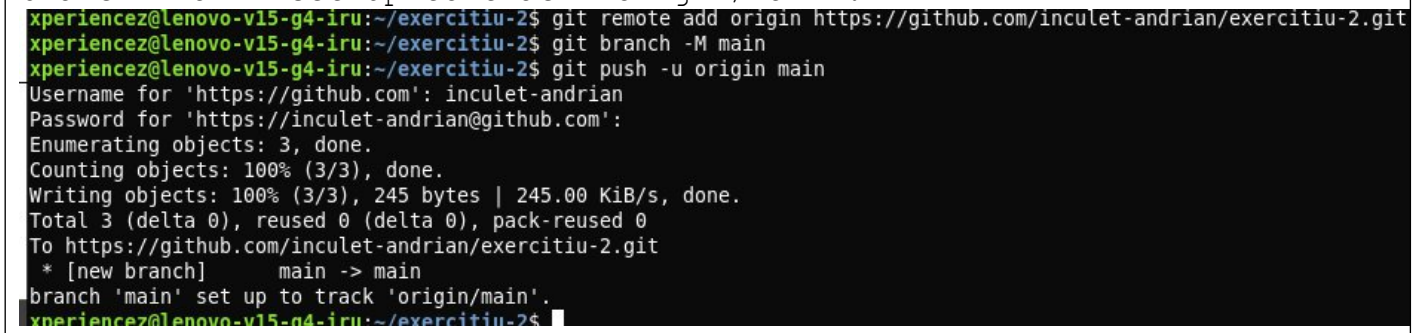


Comanda:

```
git remote add origin https://github.com/inculet-andrian/exercitiu-
2.git
git branch -M main
git push -u origin main
```

Output:

```
Username for 'https://github.com': inculet-andrian
Password for 'https://inculet-andrian@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 245 bytes | 245.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/inculet-andrian/exercitiu-2.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```



A11. Verificarea finală (4 min)

Deschide în browser cele două depozite și verifică: exercitiu-git are README.md, salut.py și .gitignore, iar în istorie (linkul cu numărul de commits) cel puțin 4 commit-uri; exercitiu-2 are README.md. Ambele sunt Public.

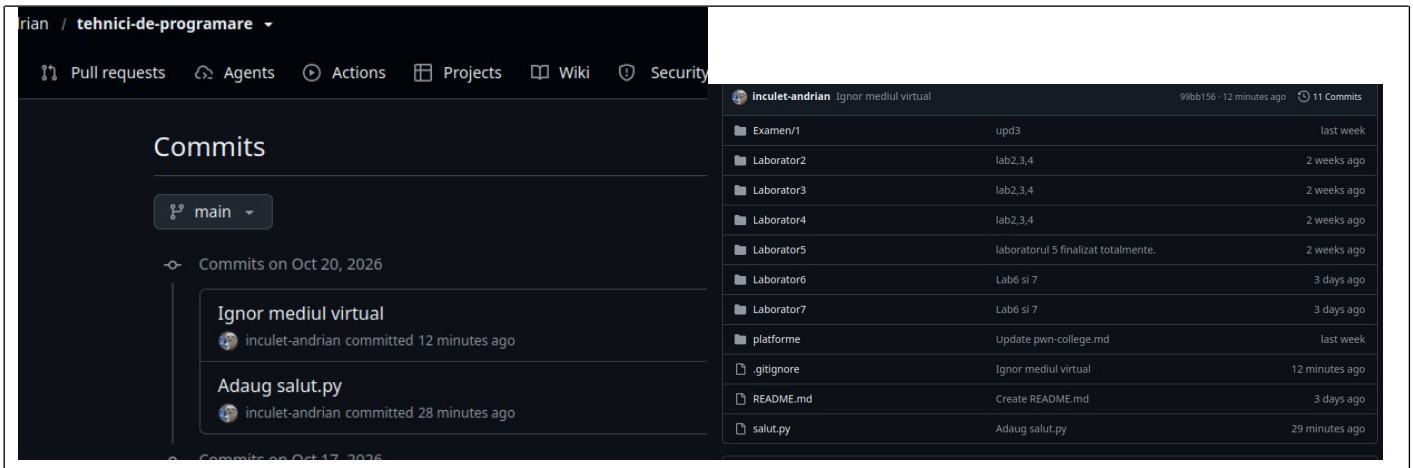


Fig 1.3, 1.4 – Commit history si fisierele create (in primul repository)

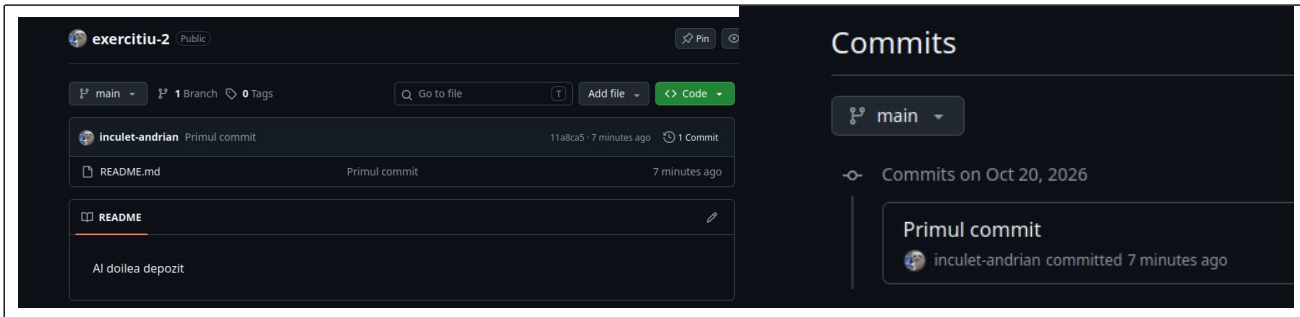


Fig 1.5, 1.6 – Commit history si fisierele create (in al doilea repository – exercitiu-2)